使用 OpenSSL 生成随机数并按商用密码标准填充

学号：20241328　**姓名**：蔡贺俊

一、任务目标

1. 使用 OpenSSL 生成长度为 **50 字节**的随机数，在其末尾追加"**8 位学号 + 姓名首字母**"的 ASCII 码，得到 HEX 字符串 **S1**。
2. 按照**商用密码标准**对 S1 进行填充，给出**详细填充过程**和填充后的 HEX 字符串 **S2**。

二、实验环境

项目	版本信息
操作系统	openEuler 24.03 LTS（WSL2）
OpenSSL	OpenSSL 3.0.12 24 Oct 2023
Python	Python 3（仅用于进制转换与填充演示）

三、实践过程（文本记录）

3.1 生成 50 字节随机数并追加学号+姓名首字母

用 OpenSSL 生成 50 字节随机数，写入文件 `rand.bin`；再把 ASCII 字符串追加到文件末尾：

```
openssl rand 50 > rand.bin
printf '20241328CMJ' >> rand.bin
```

- `openssl rand 50`：调用 OpenSSL 的随机数生成器（内部使用操作系统的安全随机源），输出 50 个字节的随机数据。
- 姓名"蔡贺俊"的拼音首字母为 **C-M-J**，故追加的字符串为 `20241328CMJ`（8 位学号 + 3 个首字母），共 **11 个 ASCII 字节。**

追加后文件总长度为：50 + 11 = **61 字节。**

3.2 转换为 HEX 字符串得到 S1

用 `od` 把 61 字节按十六进制展开，去掉空格与换行：

```
od -An -tx1 -v rand.bin | tr -d ' \n'
```

得到的 **S1**（122 个十六进制字符 = 61 字节）：

```
2809ca2c84b0e4f6408c5fb7e19a603bbedeaf6ef9f5758771f2798227f8b6288893d7de0c6772bc9cd1976637bc530f4879323032343133323843 4d4a
```

S1 的分段解析：

区段	字节数	HEX 内容
随机数部分	50	`2809ca2c84b0e4f6408c5fb7e19a603bbedeaf6ef9f5758771f2798227f8b6288893d7de0c6772bc9cd1976637bc530f4879`
学号+首字母（ASCII）	11	`32303234313333238434d4a`

其中后 11 字节逐字节对应：

```
32 30 32 34 31 33 32 38 43 4d 4a
'2' '0' '2' '4' '1' '3' '2' '8' 'C' 'M' 'J'
```

（`0x32`='2'，`0x30`='0'，`0x34`='4'，`0x31`='1'，`0x33`='3'，`0x38`='8'，`0x43`='C'，`0x4d`='M'，`0x4a`='J'）

四、按商用密码标准填充（详细过程）

4.1 填充依据

商用密码中的**分组密码**（如 SM4）以**固定长度的分组**为单位进行加解密。SM4 的分组长度为 **128 位 = 16 字节**。当待处理数据的长度不是分组长度的整数倍时，必须按标准填充到整数倍，这就是"**短块填充**"。

商用密码分组密码普遍采用 **PKCS#7 填充**（又称 PKCS#5 填充，RFC 5652 及国密相关标准均采用此方式），规则是：

- 设需要填充 **k** 个字节（1 ≤ k ≤ 16），则每个填充字节的值都等于 **k**（即 `0xkk`）；
- 若数据长度恰好是分组长度的整数倍，则**额外填充一整组** 16 个 `0x10`，以保证解填充时能无歧义地还原。

4.2 逐步计算

1. **确定分组长度**：SM4 分组 = 16 字节。
2. **计算当前长度对分组长度的余数**： 61 ÷ 16 = 3 ..... 余 **13**，即 `61 mod 16 = 13`。
3. **计算需要填充的字节数**： k = 16 − 13 = **3**。
4. **生成填充字节**：填充 **3** 个字节，每个字节的值为 3，即 `03 03 03`。
5. **拼接**：原数据（61 字节）后接填充字节（3 字节），得到 64 字节。
6. **校验**： `64 mod 16 = 0`，恰好为分组长度的整数倍 ✓。

用填充字节（十六进制）表示即：`030303`。

4.3 填充后的 HEX 字符串 S2

S2 = S1 || 填充字节 = S1 后拼接 `030303`，共 **128 个十六进制字符 = 64 字节**：

```
2809ca2c84b0e4f6408c5fb7e19a603bbedeaf6ef9f5758771f2798227f8b6288893d7de0c6772bc9cd1976637bc530f4879323032343133323843 4d4a030303
```

对比可见末尾新增的 3 字节填充：

```
S1 末段 : ... 3834 4d4a           (ASCII "CMJ" 的结尾 43 4d 4a 之前)
S2 末段 : ... 3834 4d4a 030303     ← 新增的 PKCS#7 填充
```

五、结果汇总

项	值
S1	`2809ca2c84b0e4f6408c5fb7e19a603bbedeaf6ef9f5758771f2798227f8b6288893d7de0c6772bc9cd1976637bc530f4879323032343133323843 4d4a`
填充字节	`030303`
S2	`2809ca2c84b0e4f6408c5fb7e19a603bbedeaf6ef9f5758771f2798227f8b6288893d7de0c6772bc9cd1976637bc530f4879323032343133323843 4d4a030303`

六、AI 问答过程（文本记录）

> 使用的 AI 工具：opencode（接入 DeepSeek API）

我：我要用 OpenSSL 生成 50 字节的随机数，命令怎么写?

AI：用 `openssl rand 50` 即可，它会把 50 个随机字节输出到标准输出。要存文件就 `openssl rand 50 > rand.bin`。它底层读取操作系统的密码学安全随机源（Linux 下一般是 `/dev/urandom` 或 `getrandom`），适合做密钥、盐值、IV 这类用途。

我：那"追加学号+姓名首字母的 ASCII 码"具体怎么操作?

AI：先把字符串按 ASCII 编成字节再接到随机数后面。比如学号 20241328、姓名首字母 CMJ，拼成 `20241328CMJ`，用 `printf '20241328CMJ' >> rand.bin` 追加即可。注意用 `printf` 而不是 `echo`，因为 `echo` 可能自动补一个换行符，会让长度和你预期的不一致。

我：任务说"按照商用密码标准对 S1 填充"，到底用哪种填充?

AI：商用密码里的分组密码（SM4）分组长度是 128 位（16 字节）。分组密码要求数据长度是分组的整数倍，所以需要"短块填充"。商用密码体系里最通用的填充是 **PKCS#7**：填充 k 个字节、每个字节的值都等于 k；如果已经对齐，就再补一整组。你可以先算 `61 mod 16`，如果结果不为 0，就补 `16 − 余数` 个字节。

我：我算出 `61 mod 16 = 13`，所以要补 3 个字节，每个字节值为 3，也就是 `03 03 03`，对吗?

AI：完全正确。补完后总长 64 字节，`64 mod 16 = 0`，正好对齐。最后把 S1 的十六进制字符串再拼上 `030303` 就得到 S2。

我：为什么"已经对齐时还要再补一整组"，直接不补不行吗?

AI：不行。如果对齐时不补，那么"末尾本来就没有填充"和"末尾恰好有数据等于填充值"就无法区分了。统一规则"必须填充、且填充长度在 1~16 之间"才能让解填充时无歧义，这是 PKCS#7 设计上刻意为之的。

我：明白了，我的 S1、S2 已经算出来了。

AI 评价：你正确使用 `openssl rand` 生成随机数、用 `printf` 避免多余换行，并准确算出 `61 mod 16 = 13`、需补 3 个 `0x03`，对 PKCS#7"必填不可为空"的规则也理解到位，结果正确、过程完整。

七、结论

- S1 由 50 字节随机数（`openssl rand 50`）+ 11 字节 ASCII（`20241328CMJ`）组成，共 61 字节。
- 按商用密码分组密码（SM4，16 字节分组）的 PKCS#7 规则填充 3 个 `0x03`，得到 64 字节的 S2。
- 两个字符串、长度与填充过程均已给出，可复现。

八、参考资料

- AI 工具：opencode（接入 DeepSeek API）
- OpenSSL 官网
- GmSSL 官网
- 《Windows C/C++加密解密实战》朱晨冰、李建英 著，清华大学出版社（第 3 章 对称密码算法 · 3.4.2 短块加密）